

Navigation



Advice: how to prepare for exams

The following is a short summary of the preparation helpful in learning for and scoring well in exams and projects. It's not related to a specific discipline, but rather applicable to most cognitive tests.

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

- First and foremost, ensure you are well-rested for exams. A night of crunch-learning might not be worth the lack of concentration the following day. Putting the finishing touches on a project in the early hours of the morning will most likely backfire when having to explain the project.
- Ensure you are well-hydrated and moderately well-fed during the exam; don't overeat before an exam (your energy will be taken up by digestion). Having a bottle of water and a chocolate tablet during an exam might provide you with a clarity boost - this is general advice, consider checking with your medic/nutritionist for what works best for you. (Consider reading about the [Hierarchy of needs](#) to understand how both the environment and physiology can help or hinder your cognitive performance.)
- Make sure to read the exam before you start answering; ask for clarifications if you don't understand a question. It's generally a good idea to start with answering the questions you know best - getting stuck on a question you can't answer costs time.
- Shorter and clearer answers are generally preferable to longer ones - and quicker to write. Consider practicing this important skill.
- Make sure you know how the exam is marked, how that influences your grade, the rules for taking exams (including if extra exams might cost you more!), and have a goal in mind when starting the exam - what grade do you need, do you have a make-up exam available etc.
- In the long run, it is better and easier to learn consistently than to crunch before an exam; you will retain knowledge better and for longer if you learn a little, often, rather than a lot all at once. Also, knowledge isn't properly processed until after a good sleep cycle. (See [Wikipedia: Sleep](#) for an in-depth discussion, if you're curious.)
- For projects, it is better to work a few hours every week than to crunch close to the deadline; make sure to plan your activities appropriately, and limit the time you work for. Feature creep and

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

projects, be aware of or even write down the changes you make every session; seeing your own progress is encouraging and greatly lowers the amount of willpower required to keep going.

- Starting work is the hardest part; if you find yourself having difficulty, consider starting blindly. For instance, start a project with an empty file, then write the minimal code to get it to run, then add the minimal features. Ideally, balance between a good plan and acting upon it: plan the general features of a project, start implementing those, and make minor adjustments as you go.
- Different people learn differently, be aware how you learn best. Some people learn by reading, others by listening or conversing, yet others by doing. Some people learn by performing motions - hard do to in a cognitive subject, but consider having a fidget toy or similar object while learning.
- If you find a topic - or a whole course - hard to grasp, take the effort to rephrase that information in a way you can more easily learn: write it down in your own words and highlight it, make diagrams, write programs, etc. Then, rehash that information every week, in short bursts: when you add new information, take 10 minutes to review and perhaps slightly change the old.
- Consider having a good nutrition and fitness. Besides the obvious health advantages, they can greatly improve your mental focus. Again, check with your medic.

man

Reading the manual is the first skill needed to program under GNU/Linux and similar OSes.

```
$man man
```

invokes the manual.

Arrow keys navigate the manual, / opens up the search input, q quits the manual.

```
$man find
```

looks up a specific page of the manual.

```
$man printf
```

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

For OS-related functions and general C coding, sections 2 and 3 of the manual are the most useful.

If man doesn't answer your question, look it up on the wider internet. For specific programming issues, [stackoverflow](https://stackoverflow.com) is an invaluable resource.

screen/tmux

screen and tmux are terminal multiplexers.

This very short tutorial talks about screen. The same key actions can be performed in tmux, by using its command key. The screen command key is Control-a, while for tmux, it's Control-b. However, screen and tmux use different command-line parameters.

For brevity, Control-a is shortened to C-a

screen provides multiple virtual consoles when a single console is, in fact, available.

screen can be used when having a single ssh connection to a linux machine, to get multiple command lines.

Operating screen is simple:

- `$screen`
- screen will sometimes display a welcome screen. Press enter to dismiss it.
- A console will appear. Display the current directory:
`$pwd`
- C-a c will (c)reate a new screen window, and start a new shell process inside it.
Each independent view inside screen is named a window. They are quite different from a graphical UI window; this tutorial only refers to screen windows.
- C-a p and C-a n will navigate to the (p)revious and (n)ext windows.
- C-a <number> will navigate to specific windows. The very first window started is number 0, and the next is number 1.
On some terminals, like putty, it is best to not use the numpad to enter numbers.
- If a ssh connection or terminal emulator is closed, the screen session contained within will still remain active.

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

```
$screen -ls
```

- A screen window will close only as the process inside it closes. To exit bash, press Control-d at an empty command prompt. A screen session will close only as the last window inside it closes.
- As an emergency, you can kill all your screen sessions by using `$killall screen`. This will end all the processes inside all the screen sessions you have open, resulting in data loss.

Intermediate tmux tutorial, including window splitting:
<http://lukaszwrobel.pl/blog/tmux-tutorial-split-terminal-windows-easily>

wget

wget is a powerful console-based URL downloader. Simply feed it an URL, and it will download it in the current path:

```
$wget
```

```
'http://profs.info.uaic.ro/~eugennc/teaching/cn/res/make.sh'
```

It is best to enclose the URL with ' characters. Otherwise, the console might decide to evaluate the url, instead of feeding it unchanged to wget.

Copy/paste: On GNU/Linux, simply selecting text with the mouse copies it. Middle-mouse-button pastes. putty uses right-mouse-button to paste.

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

gcc is used to make binaries from C code.

A simple usecase for gcc is:

```
$gcc hello.c -o hello.bin -lm -ggdb
```

The various tokens mean:

- gcc: the program
- hello.c: the source file
- -o hello.bin: -o specifies the output file. hello.bin is the name for the executable binary we ask gcc to create
- -lm: -l specifies a library to link. In this case, m means the math library, described by math.h. -lSDL2 would link the SDL2 base library.
- -ggdb: asks gcc to include all the debug symbols in the created binary. This allows us to use gdb (or another debugger) to debug our code.

strace

strace is a diagnostic tool that displays system calls made by a binary. It is often the first step in debugging a program.

```
strace ./hello.bin
```

strace is a wrapper for the program. It launches it, then, as it executes, it displays the system calls as they are made.

```
$cat hello.c
#include<stdio.h>

int main(void)
{
    printf("Hello!\n");
    return 0;
}

$gcc hello.c -o hello.bin -ggdb

$strace ./hello.bin
```

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

```

...
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7f66eee71000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or
directory)
open("/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=122073, ...}) = 0
mmap(NULL, 122073, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f66eee53000
close(3)                                = 0
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or
directory)
open("/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\320\3\2\0\0\0\0"...
, 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1685264, ...}) = 0
mmap(NULL, 3791264, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3,
0) = 0x7f66ee8b4000
mprotect(0x7f66ee8b4000, 2093056, PROT_NONE) = 0
mmap(0x7f66eec48000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x194000) = 0x7f66eec48000
mmap(0x7f66eec4e000, 14752, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f66eec4e000
close(3)                                = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
= 0x7f66eee51000
arch_prctl(ARCH_SET_FS, 0x7f66eee51700) = 0
mprotect(0x7f66eec48000, 16384, PROT_READ) = 0
mprotect(0x600000, 4096, PROT_READ)     = 0
mprotect(0x7f66eee74000, 4096, PROT_READ) = 0
munmap(0x7f66eee53000, 122073)          = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 13), ...}) = 0
brk(NULL)                               = 0x1da4000
brk(0x1dc5000)                           = 0x1dc5000
write(1, "Hello!\n", 7Hello!
)                                         = 7
exit_group(0)                           = ?
+++ exited with 0 +++

```

For instance, we can see the stdout write system call containing our message. Our message is immediately printed, so the

```
write(1, "Hello!\n", 7)
```

call display is broken up by stdout.

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

`gdb` is a powerful debugger. At the very least, it can set breakpoints and pause code execution, inspect variables and print stack traces.

After building a binary with `-ggdb`

```
$gcc hello.c -o hello.bin -ggdb
```

we can run it with

```
$gdb ./hello.bin
```

`gdb` launches an interactive console, where commands can be executed:

- `(gdb) file ./hello.bin`
loads a binary file, if we've accidentally run `gdb` without arguments.
- `(gdb) run`
runs the program.
- `(gdb) run arg1 arg2`
runs the program with arguments
- `(gdb) break 4`
`(gdb) b 4`
add a breakpoint at line 4.
- `(gdb) clear 4`
clears a breakpoint at line 4.
- `(gdb) print argv`
`(gdb) p argv[0]`
prints variables.
- `(gdb) continue`
`(gdb) c`
continues the execution.
- `(gdb) next`
`(gdb) n`
continues the execution until the next line is reached.
`(gdb) n 5`
continues the execution until the 5th line is reached.
- `(gdb) stepi`
`(gdb) s`
steps into the next line. If that line contains a function, `gdb` will go into its code.
- `(gdb) up`
`(gdb) down`
navigate up and down frames. If we're in a function, we can press up to see the code where that function is called.

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

prints frames, starting from the current one, up the stack.

- `(gdb) quit`
`(gdb) q`
quits gdb. C-d (Ctrl-d) works as well.
- `(gdb) set follow-fork-mode child`
tells gdb to follow the child process after a fork. This allows child-process code debugging.

gdb has a rudimentary interface, accessible with `-tui`

```
$gdb -tui ./hello.bin
```

emacs offers good integration and a better UI. Simply

```
M-x gdb
```

Intermediate gdb tutorial:

<https://www.cs.cmu.edu/~gilpin/tutorial/>

gdb fork debugging:

<https://sourceware.org/gdb/onlinedocs/gdb/Forks.html>

Makefile

Makefiles are scripts/configuration files that are used, most often, to describe how the binaries of a software project should be made (compiled/linked).

A series of advanced features are presented in the example Makefile:

- Variables: `CC=g++` sets the variable `CC` (C Compiler) to `g++`. Afterwards, the compiler call can be substituted to the variable `$(CC)`.

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

replaces the .cpp extension for each file with the .o extension.

- Intermediary binary objects: multiple source files are compiled (but not linked) into .o objects. The multiple .o objects are then linked into a single executable binary.

.cpp.o:

```
$(CC) $(GENERALFLAGS) $(CPPFLAGS) $< -o $@
```

is a rule that makes objects out of source files. The rule fires for each .cpp file; it generates an .o object name from the .cpp file name, then replaces the \$< variable with the .cpp file name, and the \$@ variable with the .o file name.

- Pre-requisites:

```
$(EXECUTABLE): $(OBJECTS)
```

```
$(CC) $(OBJECTS) -o $@ $(GENERALFLAGS)
```

```
$(LDLFLAGS)
```

specifies that, in order to be able to make the executable, all the objects need to first be made, themselves. After that requirement is satisfied, \$(CC) is used to link the objects together in a binary executable.

- Thus, when making all, the executable and objects are listed as prerequisites. make then executes the appropriate commands to satisfy all dependencies; if the rules are appropriate, this means making our binary. This example is a good general starting point for a small project, where all the source files reside in ./src
- The makefile can be adapted to a better structured project, by changing \$(wildcard ./src/*.cpp) to \$(wildcard ./src/*.cpp ./src/**/*.cpp) and beyond.

A more advanced Makefile example:

[Makefile](#)

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

Emacs is a powerful and reliable mostly-text editor. It is extremely configurable, but requires a great deal of learning.

Its basic strengths are powerful keyboard shortcuts, on-the-fly recording and replaying of macros, and rapid regexp search and replace.

- Emacs shortcuts are noted in a specific way.

`s`

means pressing `s`.

`S`

means entering capital `s`.

`C-s`

means pressing `Ctrl` and `s` at the same time.

`C-x s`

means pressing `Ctrl` and `x` at the same time, releasing both, then pressing `s`.

`C-x-s`

means pressing `Ctrl` and `x`, then keeping `Ctrl` pressed and pressing `s`.

- Launching special modes is done with

`M-x`

by pressing `alt` and `x`, releasing, then entering the name of the mode. For instance, `gdb` is launched like

`M-x gdb`

This means pressing `alt` and `x` at the same time, releasing them, then typing `gdb`, then pressing `enter`. Then pressing `enter` again, as the `gdb` minor mode asks for custom parameters.

- Opening a file in emacs is done by

`C-x-f`

Not

`C-x f`

This opens up a small dialogue (in Emacs terms, a minibuffer), where the file name can be typed. Upon typing the whole name and pressing `enter`, the file is opened in Emacs.

- `C-x-s`

saves a file.

`C-x s`

saves all files currently open and modified. Emacs makes periodic saves, and saves the old version of files with the `~` extension (the standard backup extension).

- `C-x-c`

exits Emacs

Navigation



Advice: how to prepare for exams

Tools: man

Tools: screen/tmux

Tools: wget

Tools: gcc

Tools: strace

Tools: gdb

Tools: Makefile

Tools: Emacs

searches for a string.

C-M-s

searches for a regular expression. That is Ctrl-Alt-s .

- C-r

searches backward.

- M-%

replaces a string.

- C-M-%

replaces a regexp.

- f3

starts recording a macro.

- f4

ends the recording of a macro (if currently recording one).

Otherwise, it executes the last recorded macro. Emacs macros are exceptionally powerful. As they replay the same keyboard inputs, you can call various Emacs functions, like search, word skip, and calling functions through M-x, repeatedly. It is, perhaps, Emacs' most powerful feature.

A simple .emacs and .emacs.d configuration. Install this in the home directory. i.e.: ~/.emacs and ~/.emacs.d .
[emacs.zip](#)